

Основы C#: создание консольных программ

Урок 1. Создание и запуск консольной программы

Шаг 1. Открываем терминал

Windows: нажмите Win + R, введите cmd, нажмите Enter.

Шаг 2. Создаём проект — одна команда

```
dotnet new console -o MyApp
```

Эта команда создаст папку `MyApp` с готовой программой внутри.

Шаг 3. Заходим в папку

```
cd MyApp
```

Шаг 4. Запускаем

```
dotnet run
```

В терминале появится:

```
Hello, World!
```

Всё. Программа работает. Дальше мы меняем код в файле `Program.cs` (откройте его в любом редакторе — Блокнот, VS Code), сохраняем и снова пишем `dotnet run`.

Урок 2. Точка входа — метод Main

Где начинается программа

Программа **всегда начинается с метода main**. Это как входная дверь: компилятор ищет `Main` и выполняет код внутри его фигурных скобок `{ }` сверху вниз.

```
using System;

namespace MyApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Программа запустилась!");
            Console.WriteLine("Эти две строки напечатал Main");
        }
    }
}
```

Что означает каждая строка

Строка	Простыми словами
<code>using System;</code>	Служебная строка. Дает доступ к <code>Console</code> . Просто оставляйте как есть.
<code>namespace MyApp</code>	Имя вашего проекта. Не трогаем.
<code>class Program</code>	Класс — «контейнер», в котором лежит весь код. Не трогаем.
<code>static void Main(string[] args)</code>	Точка входа. Отсюда программа стартует.
<code>string[] args</code>	Аргументы запуска. Пока не нужны, не трогаем.
<code>Console.WriteLine(...)</code>	Команда «напечатать строку на экран».

Главное правило

Весь ваш код пишете **внутри фигурных скобок Main**:

```
static void Main(string[] args)
{
    // ВСЕВаш код — здесь
}
```

Урок 3. Класс с полями и методами

Два понятия — и всё

- **Поле** — это данные (что объект *имеет*).
- **Метод** — это действие (что объект *умеет делать*).

Пример: у собаки есть имя и возраст (поля), и она умеет лаять (метод).

Пишем класс

Класс — это шаблон. Сначала описываем шаблон, потом создаём по нему объекты.

```
class Dog
{
    // ПОЛЯ — данные собаки
    public string Name;
    public int Age;

    // МЕТОД — действие собаки
    public void Bark()
    {
        Console.WriteLine("Гав! Меня зовут " + Name);
    }
}
```

Создаём объект и используем его

Внутри Main:

```
static void Main(string[] args)
{
    Dog dog = new Dog();    // 1. Создали собаку по шаблону
    dog.Name = "Шарик";     // 2. Записали в поле Name значение
    dog.Age = 3;            // 3. Записали в поле Age значение

    dog.Bark();             // 4. Вызвали метод — собака залаяла
}
```

Вывод:

Гав! Меня зовут Шарик

Правило точки

Через **точку** мы обращаемся к содержимому объекта:

```
dog.Name = "Шарик";    // записать в поле
dog.Bark();             // вызвать метод
```

Полная программа целиком (можно копировать и запускать)

```
using System;

namespace MyApp
{
    class Dog
    {
        public string Name;
        public int Age;

        public void Bark()
        {
            Console.WriteLine("Гав! Меня зовут " + Name);
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Dog dog = new Dog();
            dog.Name = "Шарик";
            dog.Age = 3;

            dog.Bark();
        }
    }
}
```

Обратите внимание: класс `Dog` написан **рядом** с классом `Program`, а не внутри него.

Урок 4. Взаимодействие полей и методов

Методы внутри класса могут **читать** поля и **изменять** поля. В этом и есть вся суть.

1. Метод читает поля

```
public void PrintInfo()
{
    Console.WriteLine("Имя: " + Name + ", возраст: " + Age);
}
```

Метод просто берёт значения полей и печатает их.

2. Метод изменяет поле

```
public void Birthday()
{
    Age = Age + 1;
    Console.WriteLine(Name + " подрос! Теперь ему " + Age);
}
```

Вызвали метод — поле `Age` увеличилось на единицу.

3. Метод считает, используя поле, и возвращает результат

```
public int GetAgeInDogYears()
{
    return Age * 7;
}
```

Метод берёт поле `Age`, умножает на 7 и отдаёт результат через `return`.

Проверяем всё вместе

```
using System;

namespace MyApp
{
    class Dog
    {
        public string Name;
        public int Age;

        public void PrintInfo()
        {
            Console.WriteLine("Имя: " + Name + ", возраст: " + Age);
        }

        public void Birthday()
        {
            Age = Age + 1;
            Console.WriteLine(Name + " подрос! Теперь ему " + Age);
        }

        public int GetAgeInDogYears()
        {
            return Age * 7;
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Dog dog = new Dog();
            dog.Name = "Шарик";
            dog.Age = 3;

            dog.PrintInfo();      // Имя: Шарик, возраст: 3

            dog.Birthday();       // Шарик подрос! Теперь ему 4

            int dogYears = dog.GetAgeInDogYears();
            Console.WriteLine("По-собачьи это " + dogYears);    // 28

            dog.PrintInfo();      // Имя: Шарик, возраст: 4 ← поле изменилось!
        }
    }
}
```

```
}  
  
}
```

Обратите внимание на последнюю строку: после `Birthday()` метод `PrintInfo()` печатает **уже новый возраст**.
Это и есть взаимодействие: поле изменилось — все методы теперь видят новое значение.

Шпаргалка: запомнить пять вещей

1. **Создать проект:** `dotnet new console -o MyApp`, зайти `cd MyApp`, запустить `dotnet run`.
2. **Программа начинается с `Main`** — весь код пишем в его скобках.
3. **Поле** = данные, **метод** = действие.
4. **Создание объекта:** `Dog dog = new Dog();`, доступ через точку: `dog.Name`, `dog.Bark()`.
5. **Методы работают с полями:** читают (`Name`), изменяют (`Age = Age + 1`), возвращают (`return Age * 7`).

Справочник терминов

Все термины из уроков — простыми словами. Если забыл, что значит слово — ищи здесь.

Терминал и проект

Термин	Что это простыми словами
Терминал	Окно, где можно вводить текстовые команды компьютеру вместо мышки. Открывается: <code>Win + R</code> → набрать <code>cmd</code> → <code>Enter</code>
Команда	Текстовая инструкция для терминала. Например, <code>dotnet run</code> — команда «запусти программу»
dotnet	Программа-помощник, которая умеет создавать и запускать проекты на C#. Все команды начинаются с её имени: <code>dotnet ...</code>
Проект	Папка с файлами вашей программы. Создаётся командой <code>dotnet new console -o Имя --use-program-main</code>
Program.cs	Главный файл проекта. Именно в нём мы пишем код. <code>.cs</code> — расширение файлов с кодом C# (си-шарп)
dotnet run	Команда: собрать программу и сразу запустить
dotnet build	Команда: только собрать программу (проверить, что нет ошибок), не запуская
Сборка / компиляция	Превращение вашего текстового кода в программу, которую умеет выполнять компьютер
Ошибка компиляции	Сообщение, что в коде опечатка или нарушение правил. Программа не запустится, пока не исправите

Слова внутри кода

Термин	Что это простыми словами
<code>using System;</code>	Служебная строка вверху файла. Даёт доступ к командам вроде <code>Console.WriteLine</code> . Просто оставляйте как есть
<code>namespace</code> (пространство имён)	Имя-ярлык для группы классов. Совпадает с именем проекта. Не трогаем
<code>class</code> (класс)	Шаблон/чертёж. Описывает, какие данные (поля) и действия (методы) будут у объектов
<code>Main</code> (точка входа)	Метод, с которого программа начинает работу . Компьютер ищет <code>Main</code> и выполняет код в его скобках <code>{ }</code>
<code>static</code>	Служебное слово. Пока просто пишем его перед <code>Main</code> и методами, вызываемыми из <code>Main</code> , не задумываясь
<code>void</code>	Означает «метод ничего не возвращает» — просто делает действие и всё
<code>string[] args</code>	Аргументы командной строки. Пока не нужны — просто часть записи <code>Main</code> , не трогаем
<code>public</code>	Означает «открыто». Поле или метод видны снаружи класса. Пока ставим <code>public</code> везде
<code>new</code>	Команда «создать объект по шаблону». Например: <code>Dog dog = new Dog();</code>

Строки кода

Термин	Что это простыми словами
<code>Console</code>	Класс для работы с окном консоли: печатать и читать текст
<code>Console.WriteLine("...")</code>	Напечатать текст и перейти на новую строку
<code>Console.Write("...")</code>	Напечатать текст без перехода на новую строку
<code>Console.ReadLine()</code>	Подождать, пока пользователь что-то напечатает и нажмёт Enter. Вернёт введённый текст
Точка с запятой <code>;</code>	Знак конца команды. Каждая команда в коде заканчивается им. Забыли — будет ошибка
Фигурные скобки <code>{ }</code>	Контейнеры для кода. В <code>{ }</code> метода лежит его код. Открывающая <code>{</code> и закрывающая <code>}</code> всегда парные
Круглые скобки <code>()</code>	Используются у методов: <code>Main()</code> , <code>Bark()</code> . Внутри — параметры
Комментарий	Текст после <code>//</code> , который игнорируется программой. Пишут пояснения для людей: <code>// здесь считаем сумму</code>
<code>+</code> (в тексте)	Склеивание строк: <code>"Меня зовут " + Name</code> превратится в <code>Меня зовут Шарик</code>

Данные

Термин	Что это простыми словами
--------	--------------------------

Термин	Что это простыми словами
Переменная	Именованная коробочка для хранения значения: <code>int age = 3;</code>
Тип данных	Какое содержимое можно положить в переменную: целое число, дробное, текст и т.д.
int	Целые числа: 3, 100, -50
double	Дробные числа: 1.5, 3.14
string	Текст в двойных кавычках: "Шарик"
char	Один символ в одинарных кавычках: 'A'
bool	Только два значения: <code>true</code> (да) или <code>false</code> (нет)
Значение	То, что лежит в переменной. У <code>age = 3</code> значение — число 3

Логика (условия и циклы)

Термин	Что это простыми словами
if	«Если». Делает код в скобках, только когда условие верно
else	«Иначе». Делает код, когда условие <code>if</code> не сработало
Условие	Проверка, которая даёт <code>true</code> или <code>false</code> . Например: <code>age >= 18</code>
switch	Выбор из нескольких готовых вариантов (как меню)
case	Один вариант в <code>switch</code> : «если день равен 1 — сделать это»
break	Стоп. В <code>switch</code> — выйти после варианта. В цикле — прервать цикл совсем
continue	Пропустить текущий повтор цикла и перейти к следующему
for	Цикл-счётчик: повторить код известно сколько раз. <code>for (int i = 1; i <= 5; i++)</code> — пять повторов
while	Цикл «пока»: повторять, пока условие верно. Число повторов заранее неизвестно
do-while	Цикл, который делает тело минимум один раз , потом проверяет условие. Подходит для меню
i++	Увеличить на 1. То же самое, что <code>i = i + 1</code>
**== **	Сравнение «равно ли». Не путать с одним <code>=</code> (записать значение)!
%	Остаток от деления. <code>10 % 3 = 1</code> (делим 10 на 3, остаётся 1). Так проверяют чётность: <code>x % 2 == 0</code>

Массивы

Термин	Что это простыми словами
Массив	Коробка с пронумерованными ячейками для нескольких значений одного типа
Элемент	Одно значение внутри массива
Индекс	Номер ячейки. Нумерация с нуля! Первый элемент — <code>arr[0]</code> , второй — <code>arr[1]</code>
Length	Свойство «сколько всего ячеек». <code>numbers.Length</code> — длина массива
new int[5]	Создать массив на 5 ячеек (все пустые, нули)
{ 10, 20, 30 }	Создать массив и сразу заполнить значениями

Методы, поля, объекты

Термин	Что это простыми словами
Метод	Действие, которое умеет делать объект. Пишется как блок кода с именем: <code>Bark()</code> , <code>PrintInfo()</code>
Поле	Данные, которые объект имеет: <code>Name</code> , <code>Age</code>
Объект	Реальная «вещь», созданная по шаблону-классу. Из одного класса <code>Dog</code> можно сделать много собак
Параметр	Данные, которые метод принимает внутрь: в <code>PrintSum(int a, int b)</code> параметры — <code>a</code> и <code>b</code>
<code>return</code>	«Верни ответ». Метод отдаёт результат наружу: <code>return Age * 7;</code>
<code>int</code> (перед методом)	Тип результата, который метод возвращает. <code>public int GetAge()</code> вернёт целое число. <code>void</code> — ничего не вернёт
Вызов метода	Запуск действия: пишем имя объекта, точку, имя метода и скобки: <code>dog.Bark()</code> ;
Точка <code>.</code>	Знак доступа к содержимому объекта: <code>dog.Name</code> — поле, <code>dog.Bark()</code> — метод
Конструктор	Специальный метод, который срабатывает при создании объекта и сразу заполняет поля: <code>new Student("Иван", 19)</code>
Свойство	То, что можно прочитать у объекта, но это не поле-коробочка, а вычисляемое значение (как <code>Length</code> у массива)

Мини-словарь «не путай»

Путают	Разница
<code>=</code> и <code>==</code>	<code>=</code> — записать значение (<code>x = 5</code>). <code>==</code> — сравнить (<code>if (x == 5)</code>)
<code>Write</code> и <code>WriteLine</code>	<code>WriteLine</code> после текста переходит на новую строку, <code>Write</code> — нет
<code>int.Parse</code> и <code>Console.ReadLine</code>	<code>ReadLine</code> всегда даёт текст. <code>int.Parse</code> превращает текст в число
Класс и объект	Класс — чертёж, объект — реальная вещь по чертежу
Поле и метод	Поле — то, что объект имеет. Метод — то, что он делает
<code>break</code> и <code>continue</code>	<code>break</code> — выйти из цикла совсем. <code>continue</code> — пропустить один повтор
